Candidate v1: a smaller overtrained looped MoE on 128 H100s

> ⚠ Warning
>
> **Historical budget scenario.** The latest request changes the allocation to 96 GPUs for 30 days. Use the revised plan and industry-workflow audit for current decisions. This document preserves the earlier 128×40 calculation; its model and R&D matrix do not fit the new allocation unchanged.

Proposed starting point: approximately **19.88B stored parameters**, **32 physical layers**, **48 executions**, **96 routed experts per layer / top-8**, and **4.5T training tokens**. The middle 16 layers are visited twice. BF16 model weights alone are approximately **39.8GB**, supporting a single-80GB-GPU serving target subject to runtime validation.

This revises the earlier 67B capacity-heavy proposal in response to the request for a smaller, overtrained model. The configuration is a **candidate to test**, not an empirically established optimum or an already working looped trainer. The JSON specification, experiment plan, and analytical calculators make every numerical choice inspectable.

Budget contract and definition of overtraining

Assume 128 H100 SXM 80GB GPUs, 16 eight-GPU NVLink nodes, and fast inter-node networking. Treat the stated **40 days as the hero window**. The proposed R&D program is approximately 8.0 additional full-cluster-equivalent days under the planning rate, with a hard cap of 10 including contingency. Data preparation also needs CPU/storage resources and may overlap the experiments.

If the 40 days instead includes everything, reserve up to 10 for R&D and recalculate the hero before experiments begin. A 20B/4.5T run does not fit unchanged into the remaining 30 days. Reduce stored capacity, the R&D allocation, or both while maintaining the required overtraining constraint; do not silently spend the same days twice.

A concrete all-in alternative is the 40-day-total specification: reserve 10 days for preparation/contingency and 30 for the hero, use **80 experts / approximately 16.65B stored parameters**, and target **3.4T tokens** (about 204 stored-parameter TPP). Its unlooped control uses 56 experts and expert width 928. This alternative must be chosen before the experiments; it is not permission to change expert count after validating a different recipe. Its throughput and mixture-capacity gates remain required.

We operationalize the user's "10× Chinchilla" request initially as $D/N_{\text{total}} \geq 200$, with stored parameters including embeddings. At 4.5T/19.88B this is **226 tokens per stored parameter**, or **11.3 times the dense 20-TPP reference**. For a looped MoE, this is a screening proxy, not proof of being 11.3× its own compute-optimal frontier. We separately report executed compute and fit the architecture's observed frontier. Chinchilla; SMELT §4.

The previous 67B/3.5T candidate had only about 52 tokens per stored parameter, despite being heavily trained relative to its active compute. This denominator difference is why both counters must be explicit.

1. Freeze a concrete architecture pair

Setting	Proposed looped hero	Unlooped control
Physical layers	32	32
Executed layers	48	32
Hidden width	2,048	3,072
Expert hidden dimension, SwiGLU	1,024	896

Setting	Proposed looped hero	Unlooped control
Experts per layer	96	72
Active routed experts	8	8
Query heads	32	48
KV heads	8	12
Head dimension	64	64
Middle span	Layers 9–24, two visits total	No loop
Residual multiplier in span	0.5 per attention and MoE write	1
Shared always-on experts	None	None
Context length	4,096	4,096
Vocabulary size	50,304	50,304

Use pre-RMSNorm, QK-Norm, RoPE base 10,000, no linear biases, untied embeddings, and a dropless token-choice router. These primitives have a concrete open precedent in **OLMoE Table 10**, including top-8 routing, SwiGLU, QK-Norm and FP32 optimizer/gradient reduction. The exact widths, expert counts and matched pair above are **our design**, not a reported SMELT or OLMoE model. OLMoE Table 10; SMELT Tables 2/9/10.

The choice of 96 rather than a very large expert pool limits stored capacity and serving memory. Test 64/96/128 experts in the small screen; 128 must not become the hero by default if its stored-size/token ratio violates the overtraining requirement. Both 96 and 72 divide by EP=8. These are hardware-compatible starting candidates, not proven expert-count optima.



Implement the repeated **whole span**, not each layer twice back-to-back. Router outputs can differ between visits. Parameters and optimizer state are shared; KV entries and saved activation contexts are visit-specific. See the SMELT review.

2. Determine compute from executions, not stored parameters

For width d, expert hidden size h, expert count E, top-k, and KV projection width d_{kv}, approximate per-physical-layer matrix counts by

$$P_{attn} = 2d^2 + 2dd_{kv}, \qquad P_{expert} = 3dh, \qquad P_{router} = dE.$$

Stored expert matrices scale with physical layers times E. Execution scales with the visit count and k. Looping does not double the training-token counter: 4.5T loss-bearing targets remain 4.5T; the repeated computation is charged through FLOPs per token. A useful forward/backward approximation is

$$F_{token} \approx 6 \sum_{\text{executions}} (P_{attn} + kP_{expert} + P_{router}) + 6Vd + 6\ell dL_{exec}.$$

The last term assumes causal triangular attention is skipped in the kernel. Segment masks reduce attention work only if the implementation actually avoids cross-document work. With variable document segments, account for the length-squared sum, not merely a document-count average. Add optimizer, recomputation and routing costs to measured runtime even when reporting them separately from model FLOPs.

`python scripts/hero_budget.py` produces the linked JSON, run CSV, mixture CSV, and the tables below. The proposed pair closely matches full model FLOPs/token and exactly matches the analytical KV-cache count; non-embedding stored counts differ by less than 1%. At small tiers, embedding differences are a larger fraction of full stored parameters and must remain in the ledger.

The H100 arithmetic ceiling is not the budget we can spend usefully. Using 989 dense BF16 TFLOP/s/GPU gives 4.375×10^{23} peak FLOPs over 40 days. **MoE sparsity is not hardware 2:4 sparsity**, so do not use the advertised sparse 1,979-TFLOP/s rate. At 250 useful model-TFLOP/s/GPU and 90% productive wall time, usable model compute is about 9.95×10^{22} FLOPs. NVIDIA reports roughly 189–279 TFLOP/s/GPU for several H100 MoE benchmarks, under mock-data/balanced-routing conditions; this supports the order of magnitude, not a guarantee for our looped model. NVIDIA benchmark table.

The 4.5T target costs approximately 9.27×10^{22} model FLOPs. Measure the actual sustained rate before launch and freeze

$$D_{hero} \leq \min(4.5\mathrm{T},\ \phi_{loss}\, 0.95\, r_{steady}\, 0.90\, T_{40days}),$$

where r_{steady} is aggregate processed-token throughput and ϕ_{loss} is the measured fraction contributing to the main loss. The initial counter assumes $\phi_{loss} = 1$ for densely packed text; recompute the cost per scored token if padding or masks make it smaller. Check $D_{hero}/N_{total} \geq 200$ as well. If throughput is too low for that condition, reduce capacity or revise the recipe before committing. Changing token horizon mid-run also changes its optimum schedule and data exposures.

3. Get both broad and high-quality data without building a new classifier first

Use the **original public Nemotron-CC release** as the web starting point. Its official Common Crawl index exposes these actual paths:

Bucket	Upstream partition	Proposed use
Broad medium	`quality=medium/kind=actual/kind2=actual/`	Main diversity supply
Medium-low	`quality=medium-low/kind=actual/kind2=actual/`	Diversity candidate; test its value
Raw low	`quality=low/kind=actual/kind2=actual/`	Explicit ablation; zero default hero weight
Medium-high	`quality=medium-high/kind=actual/kind2=actual/`	Quality/diversity balance
High	`quality=high/kind=actual/kind2=actual/`	Higher-quality real text
High synthetic	`quality=high/kind=synthetic/`	Separately tracked rephrasing/QA sources

These labels are supplied by the released partition layout, not a quality score we have invented. The records themselves list text, language, WARC ID and URL; preserve the partition label when normalizing. The official index lists **31,279 compressed JSONL objects, 10.4 TiB**. Official contents and partitions.

I have fetched the small object index and recorded its SHA256 and retrieval time in the snapshot. Corpus shards have **not** been downloaded. Reproduce or extend the pilot manifests with:

```
python scripts/prepare_cc_manifest.py
python scripts/select_cc_shards.py --quality high --count 4 --output configs/data/nemotron-cc-index/high-
        pilot.jsonl
```

```
python scripts/select_cc_shards.py --quality medium-low --count 4 --output configs/data/nemotron-cc-
        index/medium-low-pilot.jsonl
python scripts/select_cc_shards.py --quality low --count 4 --output configs/data/nemotron-cc-index/low-
        ablation.jsonl
```

The selector also writes `.urls` files. For example, after installing a download client:

```
aria2c -j 4 -x 4 --dir=data/raw/cc-high-pilot --input-file=configs/data/nemotron-cc-index/high-pilot.urls
```

Download each partition to its own directory or preserve its full relative path, so similarly named files from different quality partitions cannot overwrite each other. Four shards are an ingestion smoke test, not a token-budget sample. Count decoded tokens and keep sampling pinned shards until the required quotas are materialized. Randomize deterministically across crawls; inspect source/date/domain distributions rather than taking only the first crawl.

For code use `bigcode/starcoderdata`, and for math use `EleutherAI/proof-pile-2`. These are concrete components of Delphi's reported mixture. StarCoderData is gated; access acceptance on the user's account is a prerequisite and has not been performed here. Proof-Pile-2 is accessible as an ungated dataset. Revisions resolved for this plan are recorded in HF pins:

```
hf download bigcode/starcoderdata --repo-type dataset --revision 9fc30b578cedaec69e47302df72cf00feed7c8c4 --
        local-dir data/raw/starcoderdata
hf download EleutherAI/proof-pile-2 --repo-type dataset --revision 901a9273a770e9d4138c5ddd91802f9c5c6cdc4b --
        local-dir data/raw/proof-pile-2
```

Run the first command only with legitimate dataset access. If access is unavailable, the code-data part of this plan is blocked until a substitute is inventoried and tested; a different corpus is not silently the same recipe. Pin the OLMoE tokenizer separately using the recorded model revision, and download tokenizer assets rather than its pretrained model weights:

```
hf download allenai/OLMoE-1B-7B-0924 --revision 6d84c48581ece794365f2b8e9cfb043c68ade9c5 --include 'tokenizer*'
        'special_tokens_map.json' --local-dir data/tokenizer
```

CLI syntax follows the official download guide.

Evidence and limits: the original CC paper compares 8B/1T runs while fixing 27% non-crawl data; its HQ subset improves MMLU over DCLM, but its full pool targets longer horizons. This motivates testing quality versus diversity, not assigning 100% to HQ. Proof-Pile-2 reports about 55B tokens and StarCoderData about 250B under their source conventions. Our tokenization/dedup may reduce or change these counts. CC Tables 2–5; Proof-Pile-2 card; StarCoderData card; Delphi.

4. Materialize a feasible mixture and fixed evaluation sets

The proposed two phases use the table below: diversity for the first 80% and greater quality/code/math weight for the last 20%, synchronized with the declared cooldown. This is inspired by Nemotron Super's staged recipe; the weights and epoch caps are our hypotheses. They are not published optimal weights for this model.

Normalize into versioned records containing `text`, stable document/group ID, URL/WARC/source IDs, quality, real/synthetic kind, and parent lineage where known. Globally audit duplication across CC, synthetic text, code and mathematical text; retain the upstream processing metadata rather than destructively replacing the source. Create document-group-held-out train/dev/test corpora before repeated sampling. Preserve document masks and tokenize all candidates consistently.

For every bucket require $U_j \geq D\overline{w}_j/k_j$, using its weight averaged over both phases. The capacity table is a **go/no-go condition**, not an assertion that we have acquired those tokens. In particular, six math exposures and four code exposures are tentative high-quality caps, motivated by the finite pools and OLMo's controlled high-quality reuse; test lower caps and monitor overfitting. OLMo 3 §3.4.4.

Use 10M scored validation tokens per web-quality bucket plus code and math, with fixed domain aggregation, and a separate held-out confirmation sample. Add MMLU, ARC/HellaSwag, code and math probes with frozen prompts/scoring; expensive accuracy is not the only small-model signal. Bootstrap these evaluations and record benchmark contamination checks. The sample sizes are project choices to check against measured noise.

Plan storage explicitly: the full CC source alone is 10.4 TiB compressed; 4.5T tokens occupy 9TB at uint16 or 18TB at uint32 before indexing/replication. Use a planning allocation of **80TB shared/object storage plus hot shard caches**, then replace it with measured compression and retention needs. Save full dataset manifests and counts even when only a subset of upstream shards is retained.

5. Choose one trainer and qualify the loop extension

Use **OLMo-core** as the candidate H100/PyTorch trainer. Its README documents H100-tested dependency containers, and the MoE systems report explains EP, distributed optimizer state, recomputation and capacity costs. Neither establishes that our new repeated-layer graph works unchanged. The starting commit is pinned in trainer-pin.json; it is a research starting point, not a certified release. Official repository; MoE systems report.

Start with BF16 rather than adding a low-precision research axis. Use FP32 master/optimizer/gradient accumulation, fused attention and grouped expert GEMMs. Use EP=8 within a node, 16 expert-replication groups across nodes, TP=PP=1 initially. Dense parameters and expert parameters have different synchronization groups; do not derive global data batch solely from the expert-replica count. At 128 ranks, one 4,096-token sequence/rank and eight accumulation steps gives 4,194,304 global tokens.

At this smaller capacity, the memory margin is substantially better than the superseded 67B candidate. Still measure peak activation/dispatch/recompute memory; parameter sharding arithmetic is not a memory guarantee. Keep the repeated block local to the same pipeline stage. Only introduce PP or TP after measuring a reason, and requalify any changed topology.

Qualification tasks before scientific runs:

1. Compare shared-module execution with an explicit eager reference: forward outputs and gradients equal summed visit contributions; no duplicated optimizer states.
2. Verify checkpoint recomputation reuses the correct visit-specific state, and normalized auxiliary loss does not unintentionally grow by 1.5× with executed depth.
3. Compare one update with EP disabled versus EP=8 on an identical batch, under declared reduction tolerances.
4. Save/resume model, optimizer, scheduler, RNG and sampler; verify the next data IDs and short trajectory.
5. Run 8→32→128 GPUs, then at least 10,000 steps at peak LR. At the proposed 4,194,304-token batch, 10,000 steps consume about 41.94B processed tokens; charge this engineering run to the separate 64-GPU-day qualification reserve, not to the 10B size holdout below. Measure actual loss, routing, peak memory, throughput and checkpoint stalls.
6. Run generation with separate per-visit KV caches; compare cached versus uncached outputs and measure batch-1/batch-8 latency. The model is not “usable” merely because its weights fit one GPU.

The implementation work is required: no looped OLMo-core trainer is implemented or GPU-validated by this planning repository. This plan has not executed H100 training or serving benchmarks. These checks make “reliable trainer” a finite qualification task rather than an assumption.

6. Run a bounded reference search and data screen

At S0, for both architecture families, compare reference peak LRs $\{3, 6, 12\} \times 10^{-4}$, token batches $\{0.5, 1, 2\}$ million (using exact powers-of-two counts in the JSON), and decay $\{0.05, 0.1, 0.2\}$. Use coordinate refinement rather than a full Cartesian product: 14 S0 jobs plus six S1 LR-calibration jobs, each with 10B-token endpoints. Confirm the selected region with fresh seeds rather than reporting only its minimum observed loss.

Initial hero optimizer seed: AdamW, LR 2×10^{-4}, decay 0.1, $\beta_1 = 0.9$, $\beta_2 = 0.95$, epsilon 10^{-8}, clip 1.0. The Adam defaults and auxiliary coefficients 0.01/0.001 have OLMoE precedent; the hero LR is an unvalidated proposal. Log the layer/visit reduction convention before transferring those auxiliary coefficients. Add four S0 long-horizon LR-bracket jobs at 160B tokens (half and twice the reference-predicted LR, for both families), alongside the center point. Use these to estimate a horizon trend, then validate it at S1; the 10B reference sweep alone cannot identify a 4.5T optimum. Expand the bracket if an edge wins, within contingency. The reference search and high-TPP tests must replace unsupported LR/batch transfer assumptions. OLMoE Table 10; Power Lines; Fantastic Optimizers.

Screen **eight data recipes × two initialization seeds × 20B tokens** at the qualified S0 baseline: natural capped weights; proposed weights; remove medium-low; replace medium-low by raw low; boost high; remove synthetic; boost code; boost math. This is a targeted screen, not enough data to claim the global optimum of an arbitrary mixture-response surface. Confirm finalists at S1 under target exposure, including the quality-focused final phase.

For target-like repetition, downscale each bucket's proxy pool by D_p/D_{hero} before sampling. Carry the same bucket caps and fixed validation objective. This follows the exposure-matching logic documented by Marin. Marin data pipeline.

7. Determine depth and experts before freezing the main ladder

Allocate eight S1-equivalent 20B-token jobs to depth/width and expert screens. Test physical depths 16/20/24 and expert pools 64/96/128 under two middle visits; adjust width/FFN sizes and token counts to the stated budget and report any stored-parameter/KV mismatch. Keep top-8 fixed initially to isolate pool capacity. Shortlist based on loss, code/math scores, routing, memory and throughput, then retune locally if the shape changes.

The exact hero dimensions are the **default candidate that survives unless these measurements overturn it**. If the expert or depth choice changes, rerun the analytical counter, restore a matched unlooped control and update the ladder. Do not keep the old fit and merely relabel a new architecture. SMELT's 200M-scale span/depth/count sweeps motivate these tests but do not prove the same optimum at our horizon. SMELT §§3.3–3.5 and limitations.

8. Fit both families over three size/horizon tiers

Tier	Physical depth	Loop width / expert hidden	Baseline width / expert hidden	Endpoint tokens
S0	12	512 / 256	768 / 224	10B, 40B, 160B
S1	20	1,024 / 512	1,536 / 448	30B, 120B, 480B
S2	24	2,048 / 1,024	3,072 / 896	20B, 60B, 180B

Keep family-specific experts at 96/72 after selection, or regenerate this table if screening changes them. Train a stable prefix with separate **20%-duration cooldown branches** for the three horizons. This uses 18 endpoint observations but only six stable trunks; bootstrap by trunk, and charge the prefix once. WSD branching is supported by SMELT/Marin methodology, but this relative 20% cooldown is our adaptation to match the hero schedule, not SMELT's fixed 10B branch length.

Fit separate per-family loss surfaces on common validation data, initially $L(F, D)$ under each fixed structural recipe. The expert screen separately tests capacity at near-fixed active compute; do not pool different expert choices into an unidentified six-parameter sparsity law. Record full stored size and measured F for every run. More checkpoints do not create more independent runs.

Fresh holdouts: after freezing fits, run both S1 families with a new paired seed at **720B tokens**, independent of the fitting trunks. This is close to the desired stored-parameter TPP for that tier and directly tests long-horizon transfer. Also run the selected full 32-layer hero shape at 10B with a complete short-horizon schedule. The latter checks size/runtime transfer; it cannot certify trillions-of-token stability. If holdouts change the recipe, they become development data and require a new confirmation budget.

9. Freeze a wall-clock and serving decision, not just a loss winner

Proposed project gate: at least **5% predicted time-to-equal-loss savings**, with positive paired evidence under run-cluster uncertainty; no more than a one-point regression on critical tasks, interpreted with evaluation uncertainty; loop throughput at least 90% of control; peak memory below 72GiB; and a working cached-generation path with the single-GPU serving target. These thresholds are our acceptance policy, not universal constants.

The key conversion is

$$\frac{t_{loop}}{t_{base}} \approx \frac{C_{loop}/C_{base}}{R_{loop}/R_{base}},$$

where R is useful model-FLOP/s at the respective equal-quality runs. A 10% arithmetic saving and 15% lower achieved rate is a wall-clock loss. This is why SMELT's reported compute gains cannot simply be added to the H100 budget as free speedup.

10. Hero schedule and operational record

Freeze the measured horizon before launch, nominally 4.5T, with 1% token warmup, stable LR until 80%, then cosine decay to 2×10^{-6} over the final 20%. Apply the declared quality-focused data phase at 80%. This is an initial schedule to validate, inspired by published staged training and WSD rather than a claimed optimal annealing fraction.

Log main NTP validation separately from auxiliary losses; router load/overflow and per-visit expert overlap; per-layer residual norms; gradient/update norms; throughput and memory. Evaluate common held-out losses every 0.5% progress, the task suite every 2%, and after material interventions. Benchmark full-state checkpoint cost, then target roughly two-hour recovery points with a bounded retention policy. Reserve object storage for optimizer checkpoints, not just BF16 weights.

The overtraining gate is part of model usability. So are cached inference correctness, tested latency, tokenizer compatibility and final checkpoint evaluation. SFT/RL and long-context extension are separate budget items unless explicitly reserved inside the 40-day window; they do not appear for free after this pretraining run.

Evidence ledger: which parts are established?

Decision	Existing evidence	What remains our experiment
Two visits of middle half	SMELT Tables 3–5: selected span/count under matched budgets	Our complete architecture, data, trainer and much longer horizon
Pay for loops by matching budgets	SMELT §3.2 matches FLOPs, stored capacity, cache	Our 96/72 expert pair and runtime

Decision	Existing evidence	What remains our experiment
Smaller, overtrained model	Dense Chinchilla reference plus user serving goal; SMELT has its own MoE frontier	Stored TPP≥200 is a proxy; fit the actual loss/serving frontier
Reuse existing quality labels	Official CC five-tier object layout; CC Table 5 controlled data comparison	The optimal weight of medium-low/low at our horizon
CC + code + math starting sources	Delphi's reported source families; accessible upstream cards/pins	These proposed weights and epoch caps
Target-like proxy exposure	Marin data pipeline; OLMo constrained mixture optimization	Rank correlation for our proxies
Top-8/drop-free MoE and optimizer seed	OLMoE architectural and hyperparameter ablations	New depths, sharing, pool size and actual LR
H100 trainer choice	OLMo-core H100 container testing and published MoE systems design	Repeated-module EP/checkpoint/cache qualification
Separate fit and holdouts	AI2, Marin and SMELT empirical ladder designs	Our forecast error and seed uncertainty
Planning rate 250 TFLOP/s	NVIDIA H100 MoE benchmark order of magnitude, with stated conditions	Actual real-data rate on this cluster

Derived candidate accounting

Quantity	Looped candidate	Unlooped control
Unique stored parameters (B)	19.875	20.097
One physical traversal active body (B)	1.953	2.876
Executed matrix-equivalent count (B)	2.929	2.876
Training model FLOPs/token (G)	20.606	20.599
KV bytes/token in BF16	98,304.000	98,304.000

These are analytical projections from the candidate JSON, not profiler measurements. FLOPs assume 4,096-token full causal contexts with skipped masked triangles, no activation recomputation or optimizer overhead, and every-layer SwiGLU MoE. Stored parameter counts assume per-channel QK norm and untied embeddings. The matrix-equivalent count is an arithmetic counter, not a unique-parameter count.

Throughput sensitivity

Sustained model TFLOP/s/GPU	Productive compute over hero window	Supported looped tokens
150	5.972e+22	2.90T
200	7.963e+22	3.86T
250	9.953e+22	4.83T
300	1.194e+23	5.80T
350	1.393e+23	6.76T

The scenarios use the GPU count, hero duration and productive wall-clock fraction from the selected JSON. They are capacity projections, not measured rates.

Proposed mixture and capacity requirements

Bucket	Phase 1	Phase 2	Total exposures (B tokens)	Epoch cap	Minimum unique pool (B tokens)
cc_medium_actual	25%	15%	1035.0	1	1035.0
cc_medium_low_actual	10%	0%	360.0	1	360.0
cc_low_actual_ablation_only	0%	0%	0.0	1	0.0
cc_medium_high_actual	15%	10%	630.0	2	315.0
cc_high_actual	20%	30%	990.0	3	330.0
cc_high_synthetic	10%	15%	495.0	2	247.5
code_starcoderdata	15%	20%	720.0	4	180.0
math_proof_pile_2	5%	10%	270.0	6	45.0

Weights and caps are proposals for testing, not published optimal settings. Totals assume 4.50T tokens and an 80/20 phase split. All minimum pools must be checked after our tokenizer, global deduplication, decontamination and splits.

Proposed pre-hero GPU ledger

Experiment stage	Projected GPU-days
trainer qualification reserve	64.0
reference LR/batch/decay search: 14 S0 + 6 S1 runs x 10B	18.3
long-horizon LR brackets: 4 S0 x 160B	28.5
data screen: 8 recipes x 2 seeds x 20B	14.3
shape/expert screen: 8 x 20B at S1	32.3
core ladder with shared stable trunks	533.1
fresh paired S1 high-TPP holdouts: 2 x 720B	290.5
fresh finalist replication: 4 x 40B S1	32.3
hero-shaped holdout/rehearsal: 10B	10.6
Total	**1023.8**

Pre-hero estimates use the same 250 model-TFLOP/s/GPU and 90% duty assumptions; small jobs can be slower. The hard cap is 1,280 GPU-days, including contingency. Stop or prune before exceeding it.

Files to execute or review first

1. Review candidate.json, especially hardware assumptions, overtraining denominator and unvalidated hyperparameters.
2. Run the CC manifest tools and establish code-data access; materialize/count pilot pools before full download.
3. Qualify the pinned trainer and loop extension. Use experiments.json as the work queue, not as a training launcher.
4. Regenerate arithmetic and the GPU ledger with `python scripts/hero_budget.py`. The current projection is around 1024 GPU-days, approximately 8.0 cluster-days, below the 1,280 GPU-day cap **only at the assumed performance**.

5. Execute the bounded reference/data/architecture screens, then the frozen ladder and fresh high-TPP holds. Launch the hero only after the measured gate passes.

The preceding 67B proposal is retained as a clearly superseded JSON artifact; it is not the current recommendation. The next implementation task is the trainer/data pipeline, not another unconstrained architecture catalogue.